DOCKER: DEV + PROD-STACK

Образ, шари, кеш, контейнер, compose і healthcheck

Заняття №5

Node.js Pro

Лектор
Кирило Орлов

ПРОБЛЕМА: «НА МОЇЙ МАШИНІ ПРАЦЮЄ»

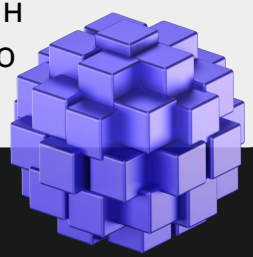
у тебе Node 22, у колеги 18, на сервері 20 — і три різні npm

«працює локально» тримається на node_modules, яких ніхто не відтворить

прод відрізняється від dev: інша ОС, інші env, інший запуск

новий колега пів дня «піднімає проєкт» замість писати код

Docker пакує застосунок разом із середовищем: один образ без змін їде в dev, CI і прод. Сьогодні збираємо його руками — від Dockerfile до compose-стека.



БЕЗ DOCKER

у тебе Node 22
у колеги Node 18
на проді Node 20
↓
три різні поведінки

З DOCKER

Dockerfile
↓ docker build
ОБРАЗ: node+deps+код
↓ docker run
dev = CI = прод

«ФОТО ПРОТИ ЗЙОМКИ»: ОБРАЗ І КОНТЕЙНЕР

ЧАСТИНА DOCKER	АНАЛОГІЯ (ФОТО)
• образ (image) • контейнер • шар (layer) • volume • docker build / docker run	• негатив — незмінний, з нього друкують • відбиток — зіпсував? друкуєш новий • шар емульсії — поверх попереднього • — переживає відбиток • проявити плівку / надрукувати відбиток

Де аналогія ламається: з одного негатива друкують скільки завгодно відбитків — і саме тому контейнер має бути одноразовим. Усе, що має пережити перезапуск, живе у volume, а не всередині.



СТЕК НА СЬОГОДНІ: IMAGE · LAYERS · CONTAINER · COMPOSE · HEALTHCHECK

Перша половина — образ	Друга половина — стек
образ ≠ контейнер: шаблон і процес	compose — сервіс + Postgres однією командою
шари й кеш — чому порядок рядків вирішує	volumes — дані переживають down
той самий код: 1.14 ГБ чи 251 МБ — вирішує Dockerfile	healthcheck — depends_on, що справді чекає
.dockerignore + контейнер зблизька: logs, exec, stop	override — dev-режим · фінал: секрети й non-root

НАСКРІЗНА НИТКА

Усе, що ти писав досі, тепер їде в контейнері. З цієї лекції й до кінця курсу код виконується всередині того образу, який зберемо сьогодні.

ЩО РОБИТЬ docker build — ШІСТЬ КРОКІВ

01. Контекст

усе з теки летить демону — тому .dockerignore обов ′язковий

02. FROM

базовий образ стає першим шаром; далі все кладеться поверх

03. COPY + RUN

кожна інструкція — новий шар; змінився шар — усі наступні перезбираються

04. Кеш

збігся шар — береться з кешу; тому package*.json копіюємо ПЕРШИМ

05. Образ

готовий образ = стос шарів + маніфест; тег — лише мітка на ньому

06. Метадані

USER, ENTRYPOINT, HEALTHCHECK — не шари, а інструкції для запуску

DOCKERFILE: СЕРЕДОВИЩЕ, ОПИСАНЕ КОДОМ

```
# 02-good.Dockerfile
FROM node:22-slim
ENV NODE_ENV=production
WORKDIR /app

# спершу ТІЛЬКИ маніфести: шар живе, поки не змінились deps
COPY package*.json ./
RUN npm ci --omit=dev

# тепер код: правка server.mjs не чіпає кеш npm ci
COPY . .

# в образі вже є користувач node (uid 1000)
USER node
EXPOSE 3000

# curl у slim немає — б'ємо в /health самим node
HEALTHCHECK --interval=5s --timeout=3s \
    --start-period=5s --retries=3 \
  CMD node -e "fetch('http://127.0.0.1:3000/health') \
    .then(r => process.exit(r.ok ? 0 : 1)) \
    .catch(() => process.exit(1))"

# node напряму: PID 1 сам отримує SIGTERM
CMD ["node", "server.mjs"]
```

Кожен рядок — відповідь на конкретні граблі

1 · package*.json копіюється ДО коду — кеш живе

2 · npm ci --omit=dev — відтворюваний install, без dev-пакетів

3 · USER node — процес не під uid 0

docker build → 251 МБ замість 1.14 ГБ у наївного

урок: розмір і швидкість збірки — наслідок порядку рядків

БАЗОВИЙ ОБРАЗ: ОДНЕ ПРАВИЛО

```
FROM node:22-slim          ← дефолт без роздумів
     Debian + glibc, npm і shell на місці
     той самий застосунок: 251 МБ проти 1.14 ГБ на node:22

FROM node:22               ← повний Debian: gcc, python, git
     ~1.1 ГБ ще ДО твого коду — образ для дебагу, не для прод

FROM node:22-alpine        ← лише якщо знаєш, навіщо
     musl замість glibc: нативні модулі (bcrypt, sharp)
     можуть не зібратися — виграш треба виміряти
```



ПРАВИЛО: починай зі slim. Alpine — коли зважив мегабайти проти ризику musl. Решта варіантів — за межами цієї лекції.

ШАРИ Й КЕШ: ЧОМУ ПОВТОРНА ЗБІРКА МИТТЄВА

- кожен FROM / COPY / RUN створює новий шар
- шар кешується за вмістом: не змінився — береться з кешу
- змінився один шар — УСІ наступні перезбираються
- COPY . . перед npm ci = переустановка залежностей на кожну правку
- правило: рідко змінюване вище, часто змінюване нижче
- RUN apt-get update && apt-get install — одним RUN, інакше кеш віддасть старий індекс

```
збірка №2 після правки server.mjs

FROM node:22-slim          → CACHED
COPY package*.json ./      → CACHED
RUN npm ci --omit=dev      → CACHED ★
COPY . .                   → змінився
усе нижче — перезбирається

★ найдовший крок не виконався
  взагалі — це і є виграш

правило порядку:
рідко змінюване  ↑ вище
часто змінюване  ↓ нижче
```



ІНСТРУКЦІЇ DOCKERFILE: ЩО СТВОРЮЄ ШАР

Інструкція	Для чого	Створює шар
FROM	базовий образ, початок стадії	так
COPY / ADD	переносить файли в образ	так
RUN	виконує команду під час збірки	так
ENV / WORKDIR	змінює середовище наступних інструкцій	ні (метадані)
USER / CMD / HEALTHCHECK	керують ЗАПУСКОМ, а не збіркою	ні (метадані)



```
# app/.dockerignore — що НЕ їде демону при збірці

node_modules
npm-debug.log
.git
.env
*.md


# скільки контексту реально летить демону:
#   docker build --no-cache --progress=plain -f 02-good.Dockerfile app
#
#   без .dockerignore:   transferring context: 3.69MB
#   з .dockerignore:     transferring context: 532B
#
# тека app важить 5.9M — майже все це node_modules,
# і без цього файлу вони їдуть у контекст КОЖНОЇ збірки
```

КОНТЕЙНЕР ЗБЛИЗЬКА



- docker run -d --name l5-demo -p 3000:3000 lecture5-good — образ став процесом
- -p публікує порт: без нього сервер живий, але з хоста не видно
- docker logs — stdout процесу; файлів логів усередині нема
- docker exec — команда всередині живого: id -u, ls /app
- writable-шар ефемерний: файл вмирає разом із контейнером
- docker stop = SIGTERM: обробник → exit 0; без нього 10 с і SIGKILL

ВІД DOCKERFILE ДО ПРОЦЕСУ: ЩО КОЛИ ВІДБУВАЄТЬСЯ

```
Dockerfile          текст
     ↓  docker build
контекст            усе з теки мінус .dockerignore
     ↓
шари                FROM · COPY · RUN — кожен окремо, кожен кешується
     ↓
ОБРАЗ               незмінний. USER / CMD / HEALTHCHECK — метадані, не шари
     ↓  docker run
КОНТЕЙНЕР           образ + writable-шар зверху + процес PID 1
     ↓
volume              єдине, що переживе видалення контейнера
     ↓
healthcheck         starting → healthy → (unhealthy)
     ↓
docker compose down        контейнер зник, volume лишився
docker compose down -v     зникло й це
```

«ГУРТОЖИТОК»: VOLUMES І МЕРЕЖІ

ЧАСТИНА COMPOSE	АНАЛОГІЯ (ГУРТОЖИТОК)
• контейнер • named volume • bridge-мережа compose • depends_on: condition: service_healthy	• мешканець кімнати — виселили й заселили нового • камера схову: речі лишаються, коли мешканець виїхав • коридор: сусіди чують одне одного на ім’я, не за номером кімнати • не заселяють, поки кімнату не прибрали

VOLUMES І МЕРЕЖІ: ДЕ ЖИВУТЬ ДАНІ

01. Контейнер одноразовий: усе, що всередині, зникає з ним

02. Named volume — керує Docker, переживає down без -v

03. Bind mount — тека з хоста; правка видно одразу

04. Compose створює мережу сам: сервіси бачать одне одного за іменем, не за IP

05. bind mount — тільки для dev: у проді код має бути в образі



```
# docker-compose.yml — база, придатна для CI як є
name: lecture5    # → імена lecture5-app-1 / lecture5-db-1
services:
  app:
    build: { context: ./app, dockerfile: ../02-good.Dockerfile }
    image: lecture5-good
    ports: ["3000:3000"]
    environment: { APP_MESSAGE: "привіт із docker-compose (базовий файл)" }
    depends_on:
      db:
        condition: service_healthy    # ← healthy, а не «просто стартував»
  db:
    image: postgres:16-alpine
    environment: { POSTGRES_PASSWORD: secret, POSTGRES_DB: app }
    volumes: [ "pgdata:/var/lib/postgresql/data" ]   # ← переживає down
    healthcheck:    # порт назовні db НЕ відкриває — app іде за іменем
      test: ["CMD-SHELL", "pg_isready -U postgres -d app"]
      interval: 3s
      retries: 10
volumes: { pgdata: }
```

```
# docker-compose.override.yml — dev-режим, підхоплюється АВТОМАТИЧНО
services:
  app:
    environment:
      APP_MESSAGE: "привіт з override — dev-файл підхопився сам"
    volumes:
      # код із хоста напряму: правиш server.mjs — зміна вже в контейнері
      - ./app:/app
      # анонімний volume-заглушка: хостові node_modules (macOS)
      # не затирають ті, що npm ci поставив усередині образу (Linux)
      - /app/node_modules

# dev бере ОБИДВА файли сам:   docker compose up -d
# CI бере тільки базу:         docker compose -f docker-compose.yml up -d
# що вийшло після merge:       docker compose config
```



DEPENDS_ON: ЧОМУ СЕРВІС СТАРТУЄ ЗАРАНО

depends_on без condition
чекає лише СТАРТУ контейнера, не готовності процесу

+ condition: service_healthy
чекає, поки healthcheck дасть healthy

Все одно потрібно

у застосунку однаково має бути ретрай підключення

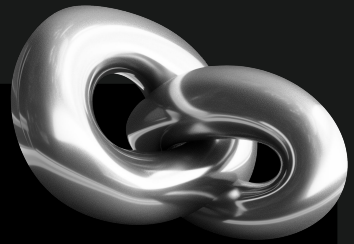
Найчастіший симптом
«Локально працює, у CI падає з ECONNREFUSED на першому запиті». Причина завжди та сама: Postgres підняв контейнер, але ще не приймає з ′єднання, а app вже пішов конектитись. depends_on без condition цього не ловить. Фікс — healthcheck із pg_isready плюс ретрай у самому застосунку: healthcheck рятує від гонки при старті, ретрай — від розриву посеред роботи.

```
# Повний цикл: від нуля до працюючого стека
docker compose up -d --wait     # повернеться, коли ОБИДВА сервіси healthy
docker compose ps               # lecture5-app-1 / lecture5-db-1 · Up (healthy)
curl -s localhost:3000/         # message = "привіт з override — dev-файл підхопився сам"

# дані переживають down (named volume):
docker compose exec db psql -U postgres -d app \
  -c "CREATE TABLE demo(id int); INSERT INTO demo VALUES (1);"
docker compose down             # БЕЗ -v: volume pgdata лишається
docker compose up -d --wait
docker compose exec db psql -U postgres -d app -c "SELECT count(*) FROM demo;"   # 1

# non-root і розміри:
docker compose exec app id -u   # 1000, не 0
docker images | grep lecture5   # good 251MB · bad 1.14GB

docker compose down -v          # прибрати все, разом із даними
```



```
# Діагностика: вісім команд, які пояснюють усе

docker images | head                    # скільки важить образ
docker history lecture5-good            # який шар скільки додав

docker run --rm lecture5-good id -u     # має бути НЕ 0
docker inspect --format '{{.State.Health.Status}}' l5-demo

# коли «локально працює, у CI ні»:
docker compose -f docker-compose.yml config    # що реально бачить CI
docker compose logs db --tail 20               # чи піднявся Postgres

# коли з'їло весь диск:
docker system df                        # хто саме розрісся
docker builder prune                    # почистити кеш збірки
```

ЩО НЕ МОЖНА КЛАСТИ В ОБРАЗ

ENV TOKEN=... — значення лишається в шарі НАЗАВЖДИ

видалив файл наступним RUN — він усе одно є в попередньому шарі

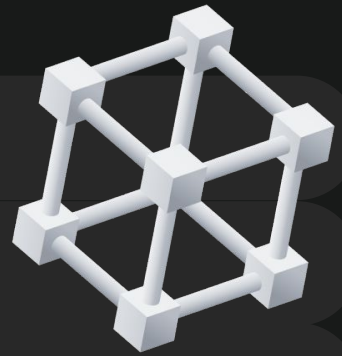
docker history показує кожну команду — включно з тією, де секрет

.env копіюється разом із COPY . . — тому він у .dockerignore

приватні ключі, .git, історія — усе це витікає з образом

правильний шлях — BuildKit secrets і змінні середовища при запуску (Лекція 28)

NON-ROOT: ЧОМУ USER — НЕ ФОРМАЛЬНІСТЬ



за замовчуванням процес у контейнері працює під root (uid 0)

пробили застосунок → отримали root у контейнері

офіційний образ node вже має користувача node (uid 1000) — досить USER node

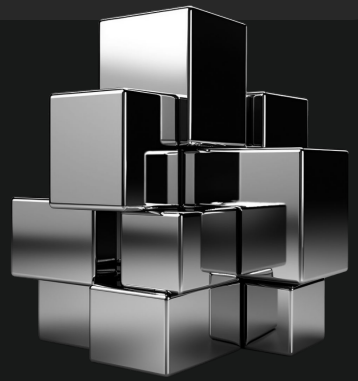
USER — після всіх RUN, що потребують прав; перевірка: docker run --rm образ id -u → не 0. Усередині Docker non-root МОЖЕ слухати 80 — але не привчайся: поза Docker (bare metal, systemd, частина K8s-рантаймів) порт <1024 вимагає CAP_NET_BIND_SERVICE. Тому слухаємо 3000 — одна звичка на всі рантайми, а reverse-proxy однаково попереду.

ЩО СВІДОМО ЛИШАЄМО НА ПОТІМ

Multi-stage — коли є крок збірки (TypeScript, бандлер): у фінальний образ їде лише зібране. Спробуєш у ДЗ; глибокий розбір — Лекція 28.

Kubernetes теж Лекція 28. Compose — не «іграшкова версія k8s», а окремий інструмент для dev і невеликого прода(не треба). Плутати їх не треба.

⚠ Compose вміє restart: unless-stopped — але це один хост: без автоскейлу, без rolling update, без кластера. Для dev і CI цього майже завжди досить; оркестрація — Лекція 28.



ПІДСУМКИ + МІСТ ДО L#6

ОБРАЗ

Образ — незмінний шаблон, контейнер — процес поверх нього

ШАРИ

порядок рядків у Dockerfile визначає і час збірки, і розмір образу

ДАЛІ

L#6 — TypeScript-декоратори й NestJS. Хочеш — ганяй їхній код у цьому ж контейнері: середовище вже описане

ДЗ #5

6 балів · Dockerfile для сервісу з L#4: single-stage на node:22-slim + non-root + healthcheck · compose з Postgres · override dev/CI · README з розмірами образів · бонус без балів: multi-stage (L#28)

COMPOSE

compose піднімає стек однією командою; volume — єдине, що переживає down

Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ